

A Fast Algorithm for Generating All Triangulations of Biconnected Plane Graphs

Tawkir Aziz Rahman

CSE, BUET

Overview

1 Introduction

2 Algorithm

The Problem:

- Generate **all possible triangulations** of biconnected plane graphs
- Previous methods fail with complex structures due to multi-edge creation
- Important applications in graphics, engineering, and chip design

Our Contribution:

- Novel algorithm with **constant time per triangulation**
- Solves the multi-edge problem using **safe root** concept
- Practical for real-world applications

Find All Triangulations Cycle

```
1: procedure FIND-ALL-CHILD-TRIANGULATIONS-CYCLE( $T_i, T, i$ )
2:   OUTPUT-TRIANGULATION( $T, T_{ir}, i$ )
3:   if  $T_i$  has no generating chords then
4:     return
5:   end if
6:   Let  $(v_b, v_{b'})$  be the leftmost blocking chord of  $T_i$ 
7:   for all  $j \geq b$  do
8:     if  $(v_1, v_j)$  is a chord of  $T_i \vee (v_1, v_j) \notin \text{GlobalChordSet}$  then
9:       add chord  $(v_1, v_j)$  to GlobalChordSet
10:      FIND-ALL-CHILD-TRIANGULATIONS-CYCLE( $T(v_1, v_j)$ )
11:      remove chord  $(v_1, v_j)$  fromz GlobalChordSet
12:    end if
13:  end for
14: end procedure
```

Find All Triangulations

```
1: procedure FIND-ALL-TRIANGULATIONS-CYCLE( $F, T, i$ )
2:    $n \leftarrow$  number of vertices for  $F_i$ 
3:    $T_{ir} \leftarrow$  FIND-SAFE-ROOT( $F, i$ ) ▷ Time  $O(1)$ 
4:   Add All Chords of the root to GlobalChordSet ▷ Time  $O(n)$ 
5:   OUTPUT-TRIANGULATION( $T, T_{ir}, i$ )
6:    $T_i = T_{ir}$ 
7:   for  $j = n - 1$  to 3 do
8:     add chord  $(v_1, v_j)$  to GlobalChordSet
9:     FIND-ALL-CHILD-TRIANGULATIONS-CYCLE( $T_i(v_1, v_j), T, i$ )
10:    Remove chord  $(v_1, v_j)$  from GlobalChordSet
11:  end for
12:  Remove All Chords of the root From GlobalChordSet ▷ Time  $O(n)$ 
13: end procedure
```

```
1: procedure OUTPUT-TRIANGULATION( $T, T_i, i$ )
2:    $T = T + T_i$ 
3:   if  $i == \text{face.length}$  then
4:     output( $T$ )
5:   else
6:     FIND-ALL-TRIANGULATION-CYCLE( $F, T, i + 1$ )
7:   end if
8: end procedure
```

Find Safe Root

```
1: procedure FIND-SAFE-ROOT( $F, index$ )
2:   FIND-SAFE-ROOT-VERTEX( $F, index$ )
3:   add chords from the safe root vertex
4:   return
5: end procedure
```

```
1: procedure FIND-SAFE-ROOT-VERTEX( $F, index$ )
2:   startindex  $\leftarrow n - 1$ 
3:   endIndex  $\leftarrow 0$ 
4:   while startindex  $>$  endIndex + 1 do
5:     if chord exists between  $V_{startindex}$  and  $V_{endIndex}$  then
6:       startindex  $\leftarrow startindex - 1$ 
7:     else
8:       endIndex  $\leftarrow endIndex + 1$ 
9:     end if
10:  end while
11: end procedure
```

Start Triangulation Process

- 1: **procedure** START-TRIANGULATION(G, k)
- 2: Label the faces $F_1, F_2, \dots, F_k$ arbitrarily
- 3: FIND-ALL-TRIANGULATIONS($F, 0$)
- 4: **end procedure**